Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 8
ПО ДИСЦИПЛИНЕ «ТИПЫ И СТРУКТУРЫ ДАННЫХ»

Графы

Вариант № 9

Студент: **Дудырев Д.С.**

Группа: **ИУ7-32Б**

Преподаватель: **Барышникова М.Ю.**

2025 г.

Отчет лаб. работа №8

Описание условий задачи

Обработать графовую структуру в соответствии с указанным вариантом задания. Обосновать выбор необходимого алгоритма и выбор структуры для представления графов. Результат выдать в графической форме.

Техническое задание

Исходные данные

- Файл, содержащий данные в формате
  - <исходная*вершина*> <*конечная*вершина> <стоимость_перехода>
- Пользовательский ввод:
  - номер команды для взаимодействия с программой;
  - Путь к файлу с входными данными (строка не более 128 символов)
  - Данные для добавление нового ребра в формате <исходная*вершина*> <*конечная*вершина> <стоимость_перехода>
  - Данные для удаления существующего ребра в формате <исходная*вершина*> <*конечная*вершина>

Выходные данные

- Таблица с результатами замеров
- таблица минимальных расстояний между вершинами
- dot файл содержаший описание графа
- png картинка – графическое представление графа

Описание реализуемой задачи

- Реализовать алгоритмы обработки графовых структур:
  - Добавление ребра между вершинами
  - Удаление ребра между вершинами
  - Нахождения минимального расстояния между вершинами
- Реализовать экспорт графовых структур в графическое представление при помощи утилиты `dot`

Способы обращения к программе

Программа запускается из командной строки при помощи команды `app.exe`.

Возможные аварийные ситуации и ошибки пользователя

Аварийная ситуация	**Код ошибки**
Данные вне корректного диапазона (отрицательные значения вершин или)	2
Ошибка чтения данных из файла	3
Ошибка выделения памяти	4
Ошибка открытия файла	5
Пустой файл	6

Аварийная ситуация	Код ошибки
Ребро уже существует (при добавлении)	7
Ребро не существует (при удалении)	8
Некорректный ввод (ввод символа)	9
Пустая строка	10
Строка длинее максимального размера (более 128 символов)	11

Описание структуры данных

Матричный граф

```
typedef unsigned int** matrixt_t;
struct graph_matr_t
{
     matrix_t vertices;   // матрица смежностей
     int vertices_count;  // количество вершин
};
```

Списочный граф

```
// Структура ребра
typedef struct edge_t edge_t;
struct edge_t
{
    int to;
    int value;
    edge_t *next;
} edge_t;

// Структура графа на списках
typedef struct graph_list_t
{
    edge_t **adj_lists;   // массив списков смежности
    int vertices_count;   // кол-во вершин
} graph_list_t;
```

Представление и обработка графа

Рассмотрим алгоритм Флойда–Уоршелла

Алгоритм обработки

Основная идея алгоритма Флойда–Уоршелла заключается в том, что для каждой пары вершин `i` и `j` в таблице `dist` последовательно перебираются все промежуточные вершины `k`. Если путь `i -> k -> j` оказывается дешевле текущего пути `i -> j`, то значение `dist[i][j]` обновляется; иначе остаётся прежним.

Форма представления

Поскольку в основе алгоритма Флойда-Уоршалла лежит динамическое программирование, то наиболее удобным способом представления будет матричное, поскольку не придется тратить ресурсы на формирование матрицы связности вершин, на основе которой будет составляться матрица минимальных расстояний между точками. Признаком отсутствия ребра является большое значение в данной работе таким числом является – `UINT32_MAX`

Описание алгоритма

Матричный граф

Добавление ребра

```
graph – граф
from_point – исходная вершина
to_point – конечная вершина
value – стоимость перехода
vertices – матрица вершин графа

graph_not_edge(graph, from_point, to_point) – проверка существует ли ребро из from_point в

Если НЕ graph_not_edge(graph, from_point, to_point))
    Вернуть EDGE_ALREADY_EXITS;
Конец если

vertices[from_point][to_point] = value;
```

Удаление ребра

```
graph – граф
from_point – исходная вершина
to_point – конечная вершина
value – стоимость перехода
vertices – матрица вершин графа
INF – условно бесконечная стоимость перехода, обозначает отсутсвие ребра

graph_not_edge(graph, from_point, to_point) – проверка существует ли ребро из from_point в

Если graph_not_edge(graph, from_point, to_point))
    Вернуть EDGE_NOT_EXITS;
Конец если

vertices[from_point][to_point] = INF;
```

Наименьшая длина меджу всеми парами

```
graph – граф в матричном виде
n – количество вершин графа
dist – матрица расстояний (результат)
graph.vertices[i][j] – вес ребра i -> j (или INF, если ребра нет)
INF – “бесконечность”


n = graph.vertices_count
dist = matrix_create(n, n)

Для i от 0 до n – 1
    Для j от 0 до n – 1
        dist[i][j] = graph.vertices[i][j]
    Конец для
```

```
Конец для

Для i от 0 до n - 1
    dist[i][i] = 0
Конец для

Для k от 0 до n - 1
    Для i от 0 до n - 1
        Если dist[i][k] == INF
            Продолжить
        Конец если

        Для j от 0 до n - 1
            Если dist[k][j] == INF
                Продолжить
            Конец если

            Если dist[i][k] > INF - dist[k][j]
                Продолжить
            Конец если

            Если dist[i][j] > dist[i][k] + dist[k][j]
                dist[i][j] = dist[i][k] + dist[k][j]
            Конец если
        Конец для
    Конец для
Конец для

Вернуть dist
```

Списочный граф

Добавление ребра

```
graph — граф в списочном виде
from — исходная вершина
to — конечная вершина
value — стоимость перехода
graph.adj_lists[from] — голова списка рёбер из вершины from
curr — текущий узел списка рёбер
new_edge — добавляемое ребро


curr = graph.adj_lists[from]
Пока curr задано
    Если curr.to == to
        Вернуть EDGE_ALREADY_EXITS
    Конец если
    curr = curr.next
Конец пока

new_edge = выделить память под edge_t
Если new_edge не задано
    Вернуть MEM_ERROR
Конец если

new_edge.to = to
```

```
new_edge.value = value

new_edge.next = graph.adj_lists[from]
graph.adj_lists[from] = new_edge
```

Удаление ребра

```
graph — граф в списочном виде
from — исходная вершина
to — конечная вершина
graph.adj_lists[from] — голова списка рёбер из вершины from
curr — указатель на указатель на текущее ребро (edge_t **)
to_delete — удаляемое ребро


curr = адрес graph.adj_lists[from]
Пока *curr задано
    Если (*curr).to == to
        to_delete = *curr
        *curr = (*curr).next
        Вернуть OK
    Конец если

    curr = адрес (*curr).next
Конец пока

Вернуть EDGE_NOT_EXISTS
```

Наименьшая длина меджу всеми парами

```
graph — граф в списочном виде
n — количество вершин графа
dist — матрица расстояний (результат)
graph.adj_lists[i] — голова списка рёбер из вершины i
e.to — конечная вершина ребра
e.weight — вес ребра
INF — “бесконечность”


n = graph.vertices_count
dist = matrix_create(n, n)

Для i от 0 до n - 1
    dist[i] = выделить память на n элементов
    Для j от 0 до n - 1
        dist[i][j] = INF
    Конец для
Конец для

Для i от 0 до n - 1
    Для e = graph.adj_lists[i]; e задано; e = e.next
        dist[i][e.to] = e.weight
    Конец для

    dist[i][i] = 0
Конец для
```

```

Для k от 0 до n - 1
    Для i от 0 до n - 1
        Если dist[i][k] == INF
            Продолжить
        Конец если

        Для j от 0 до n - 1
            Если dist[k][j] == INF
                Продолжить
            Конец если

            Если dist[i][k] > INF - dist[k][j]
                Продолжить
            Конец если

            Если dist[i][j] > dist[i][k] + dist[k][j]
                dist[i][j] = dist[i][k] + dist[k][j]
            Конец если
        Конец для
    Конец для
Конец для

Вернуть dist
```

Необходимые функции

Матричный граф

`graph_matr_t *graph_create(int points_count);` – создание графа из `points_count` вершин

- `points_count` – количество вершин в графе
  `int graph_add_edge(graph_matr_t *graph, int from_point, int to_point, int weight);` – добавление ребра `from_point -> to_point` с весом `weight` в матричный граф
- `graph` – указатель на граф
- `from_point` – индекс исходной вершины
- `to_point` – индекс конечной вершины
- `weight` – вес ребра

`int graph_remove_edge(graph_matr_t *graph, int from_point, int to_point);` – удаление ребра `from_point -> to_point` из матричного графа

- `graph` – указатель на граф
- `from_point` – индекс исходной вершины
- `to_point` – индекс конечной вершины

`matrix_t graph_vertices(graph_matr_t *graph);` – получение матрицы смежности графа (матрицы весов)

- `graph` – указатель на граф

`int graph_vertices_count(graph_matr_t *graph);` – получение количества вершин в графе

- `graph` – указатель на граф

`int graph_read_from_file(FILE *f, graph_matr_t **graph_ptr);` – чтение матричного графа из файла `f` и запись указателя на созданный граф в `graph_ptr`

- `f` – указатель на файл
- `graph_ptr` – адрес указателя, куда будет записан созданный граф

`int export_graph_matr_to_dot(graph_matr_t *graph, char *filename_dot);` – экспорт матричного графа в файл формата dot

- `graph` – указатель на граф
- `filename_dot` – имя/путь к выходному `.dot` файлу

`bool graph_not_edge(graph_matr_t *graph, int from_vert, int to_vert);` – проверка отсутствия ребра `from_vert -> to_vert` в графе

- `graph` – указатель на граф
- `from_vert` – индекс исходной вершины
- `to_vert` – индекс конечной вершины

`int graph_extend_matrix(graph_matr_t *graph, int new_count);` – расширение матрицы смежности графа до `new_count` вершин

- `graph` – указатель на граф
- `new_count` – новое количество вершин (больше текущего)

`int graph_path_value(graph_matr_t *graph, int from_vert, int to_vert);` – получение веса ребра `from_vert -> to_vert` (стоимости перехода)

- `graph` – указатель на граф
- `from_vert` – индекс исходной вершины
- `to_vert` – индекс конечной вершины

`matrix_t graph_shortest_paths(graph_matr_t *graph);` – вычисление матрицы кратчайших путей между всеми парами вершин (алгоритм Флойда–Уоршелла)

- `graph` – указатель на граф

`void graph_destroy(graph_matr_t *graph);` – освобождение памяти, выделенной под граф

- `graph` – указатель на граф

`size_t graph_memory_capacity(graph_matr_t *graph);` – оценка объёма памяти, занимаемого матричным графом (в байтах)

- `graph` – указатель на граф

Списочный граф

`graph_list_t *graph_list_create(int vertices_count);` – создание списочного графа из `vertices_count` вершин

- `vertices_count` – количество вершин в графе

`int graph_list_add_edge(graph_list_t *graph, int from, int to, int weight);` – добавление ребра `from -> to` с весом `weight` в списочный граф

- `graph` – указатель на граф
- `from` – индекс исходной вершины
- `to` – индекс конечной вершины
- `weight` – вес ребра

`int graph_list_remove_edge(graph_list_t *graph, int from, int to);` – удаление ребра `from -> to` из списочного графа

- `graph` – указатель на граф
- `from` – индекс исходной вершины
- `to` – индекс конечной вершины

`bool graph_list_not_edge(graph_list_t *graph, int from, int to);` – проверка отсутствия ребра `from -> to` в списочном графе

- `graph` – указатель на граф
- `from` – индекс исходной вершины
- `to` – индекс конечной вершины

`void graph_list_destroy(graph_list_t *graph);` – освобождение памяти, выделенной под списочный граф

- `graph` – указатель на граф

`int graph_list_vertices_count(graph_list_t *graph);` – получение количества вершин в списочном графе

- `graph` – указатель на граф

`int graph_list_path_value(graph_list_t *graph, int from, int to);` – получение веса ребра `from -> to` (стоимости перехода)

- `graph` – указатель на граф
- `from` – индекс исходной вершины
- `to` – индекс конечной вершины

`unsigned int **graph_list_shortest_paths(graph_list_t *graph);` – вычисление матрицы кратчайших путей между всеми парами вершин (через заполнение `dist` по спискам и алгоритм Флойда–Уоршелла)

- `graph` – указатель на граф

`size_t graph_list_memory_capacity(graph_list_t *graph);` – оценка объёма памяти, занимаемого списочным графом (в байтах)

- `graph` – указатель на граф

`int graph_list_read_from_file(FILE *f, graph_list_t **graph_ptr);` – чтение списочного графа из файла `f` и запись указателя на созданный граф в `graph_ptr`

- `f` – указатель на файл
- `graph_ptr` – адрес указателя, куда будет записан созданный граф

`int export_graph_list_to_dot(graph_list_t *graph, char *filename_dot);` – экспорт списочного графа в файл формата dot

- `graph` – указатель на граф
- `filename_dot` – имя выходного `.dot` файла

`int graph_list_extend(graph_list_t *graph, int new_count);` – расширение списочного графа до `new_count` вершин

- `graph` – указатель на граф
- `new_count` – новое количество вершин (больше текущего)

Тесты

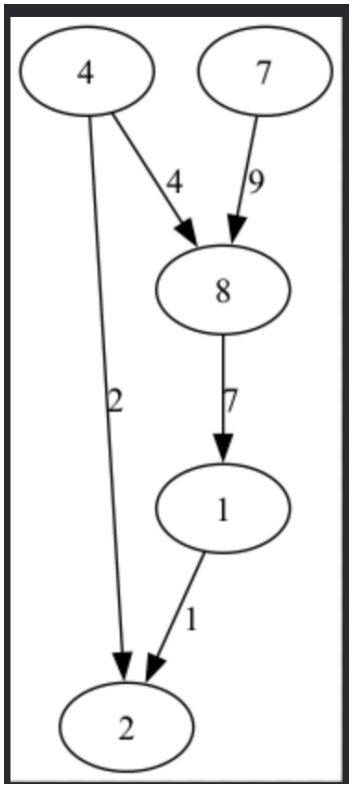
Тесты аналогичны для матричной и списочной реализаций

Негативные

1. Добавление существующего ребра
2. Удаление не существующего ребра
3. Ошибка открытия файла
4. Пустой файл
5. Ввод не числа при добавлении
6. Ввод отрицательного числа при добавлении
7. Ввод отрицательного числа при удалении
8. Ввод не числа при удалении
9. Удаление рерка, но вводиться в обратном направлении

1) Добавление существующего ребра

Ввод



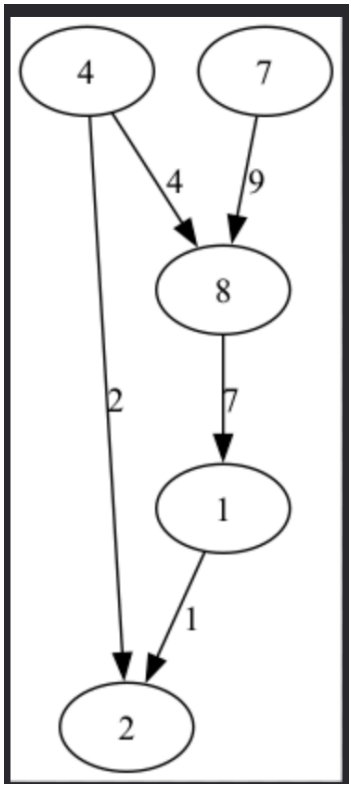
Вывод

```
Ввод в формате: <исходная_вершина> <конечная_вершина> <стоимость_перехода>
```

```
4 2 2
Ребро уже существует
```

2) Удаление не существующего ребра

Ввод



Вывод

```
Ввод в формате: <исходная_вершина> <конечная_вершина>
4 7
Ребра не существует
```

3) Ошибка открытия файла

Ввод

```
├── app.exe
├── collect_coverage.sh
├── file_generate.sh
├── func_tests
├── graph.dot
├── inc
├── makefile
├── out
├── res_list.png
├── res_matr.png
├── src
├── test copy.txt
├── test.txt
└── valg_check.sh
```

```
Введите название файла: asd
```

Вывод

```
Ошибка открытия файла
```

4) Пустой файл

Ввод

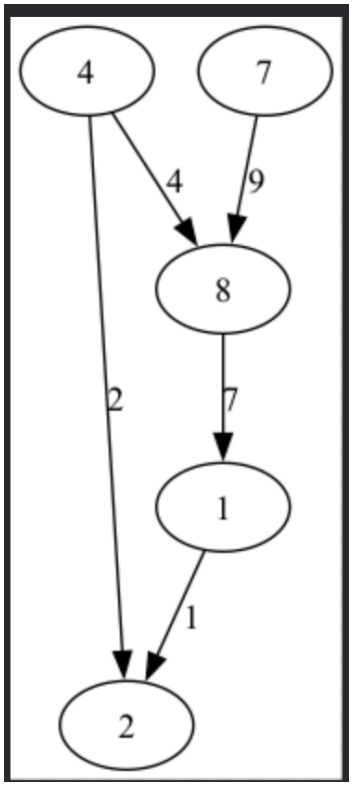
```
Введите команду: 1
Введите название файла: test.txt
```

Вывод

```
Файл пустой
```

5) Отрицательные значения при вводе

Ввод



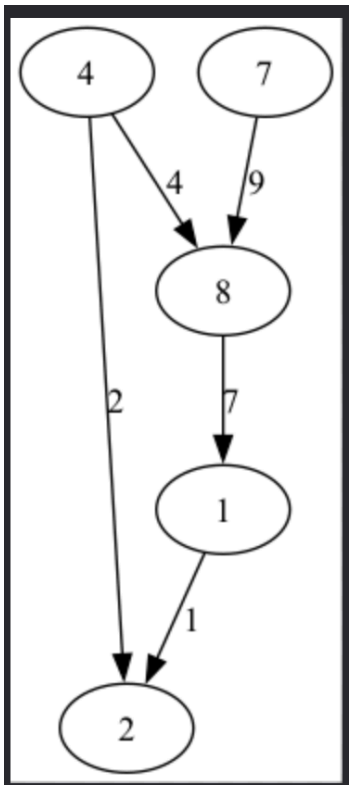

```
Введите команду: 1
Ввод в формате: <исходная_вершина> <конечная_вершина> <стоимость_перехода>
4 -5 2
```

Вывод

```
Некоректные диапазон значений
```

6) Ввод не числа при добавлении

Ввод



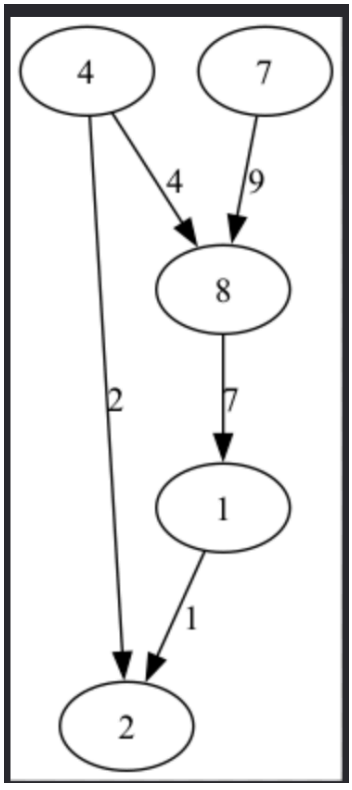

```
Ввод в формате: <исходная_вершина> <конечная_вершина> <стоимость_перехода>
3 a 9
```

Вывод

```
Некоректный ввод
```

7) Ввод отрицательного числа при удалении

Ввод



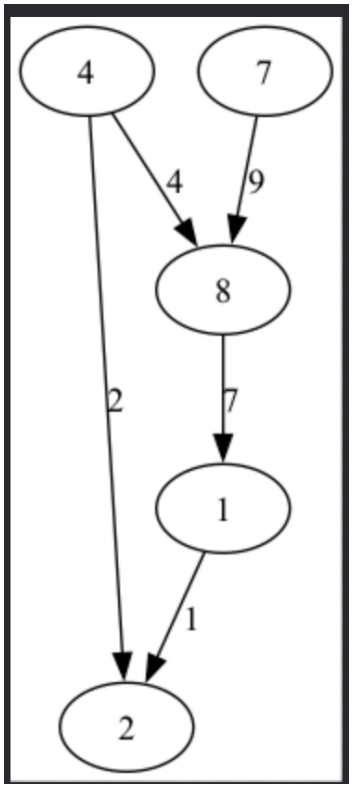
```
Ввод в формате: <исходная_вершина> <конечная_вершина>
2 -1
```

Вывод

```
Некоректные диапазон значений
```

8) Ввод не числа при удалении

Ввод



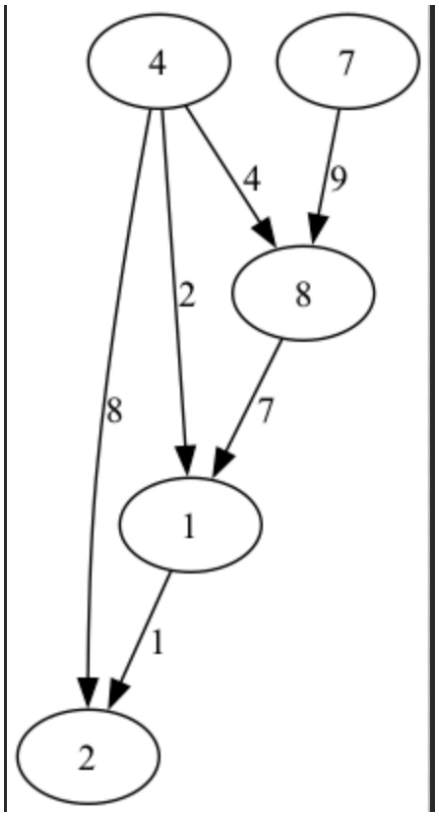

```
Ввод в формате: <исходная_вершина> <конечная_вершина>
a 3
```

Вывод

```
Некоректный ввод
```

9)Удаление ребра, направленного в другую сторону

Ввод




```
Введите команду: 2
Ввод в формате: <исходная_вершина> <конечная_вершина>
1 4
```

Вывод

```
Ребра не существует
```

Позитивные

1. Добавление первого ребра в пустой граф
2. Добавление ребра в не пустой граф
3. Расширение матрицы при добавлении ребра добавление ребра
4. Удаление существующего ребра
5. Поиск кратчайших путей

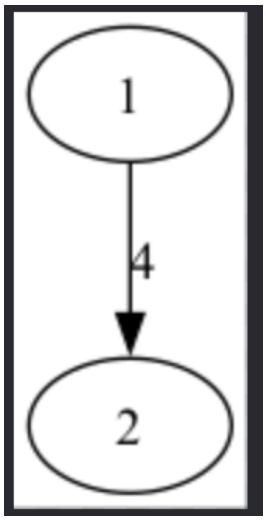
Добавление первого ребра в пустой граф

Ввод

```
Пустой граф
```

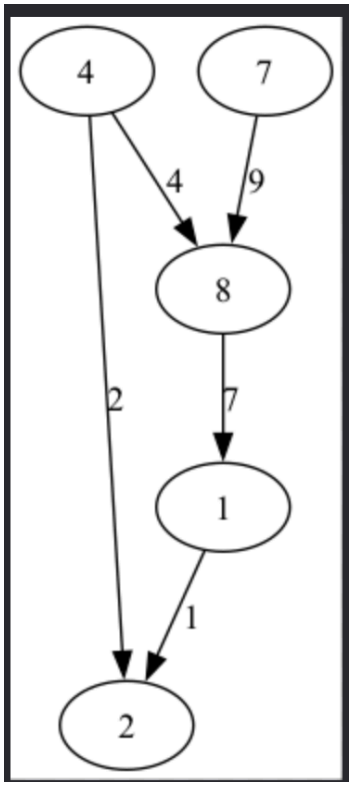
```
Введите команду: 1
Ввод в формате: <исходная_вершина> <конечная_вершина> <стоимость_перехода>
1 2 4
```

Вывод



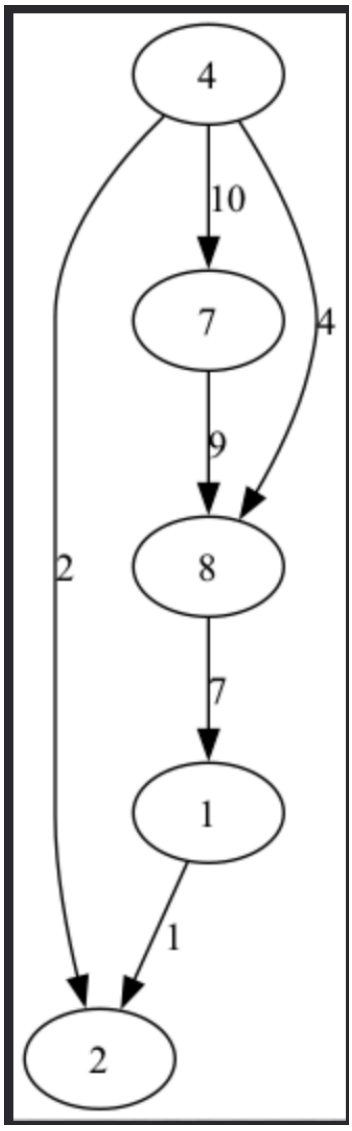
2) Добавление ребра в не пустой граф

Ввод



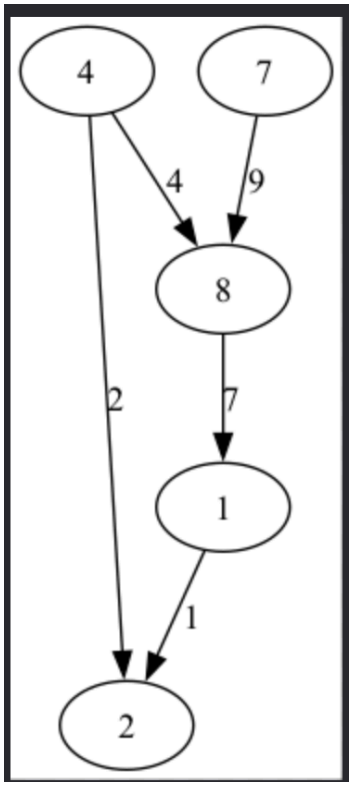

```
Введите команду: 1
Ввод в формате: <исходная_вершина> <конечная_вершина> <стоимость_перехода>
4 7 10
```

Вывод



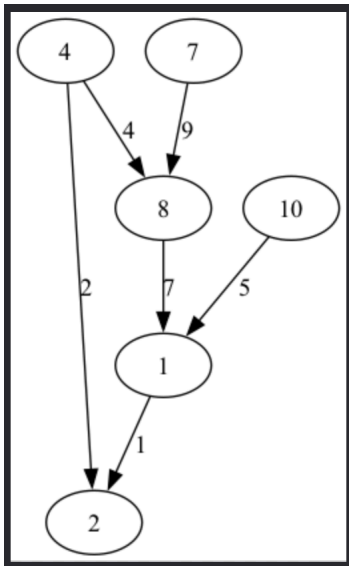
3) Расширение матрицы при добавлении ребра

Ввод



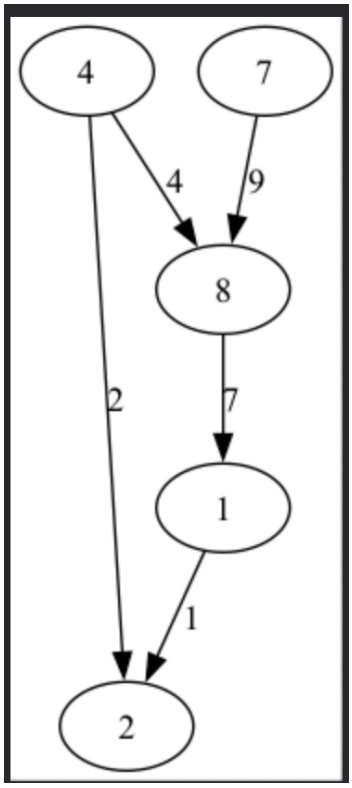
```
Введите команду: 1
Ввод в формате: <исходная_вершина> <конечная_вершина> <стоимость_перехода>
10 1 5
```

Вывод



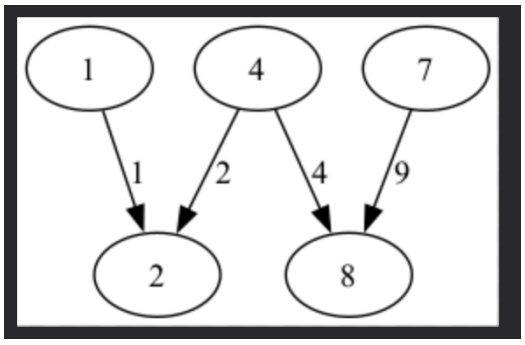
4) Удаление существующего ребра

Ввод



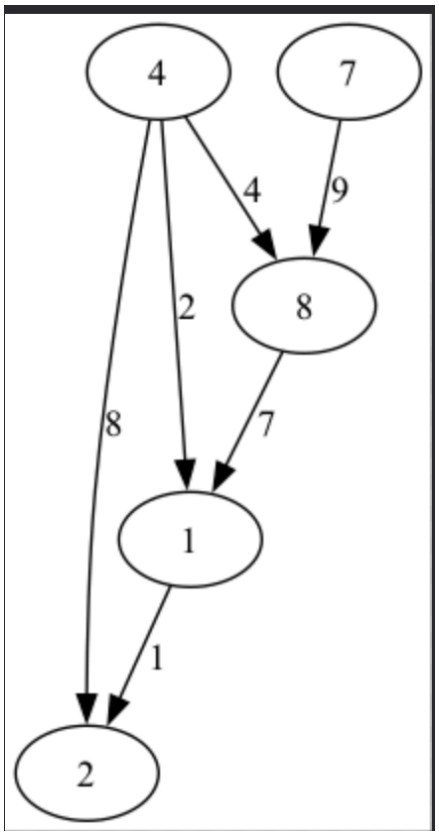

```
Введите команду: 2
Ввод в формате: <исходная_вершина> <конечная_вершина>
8 1
```

Вывод



5) Поиск кратчайших путей

Ввод




```
Введите команду: 4
```

Вывод

```
=================================================================
исх.\кон.|    1 |    2 |    3 |    4 |    5 |    6 |    7 |    8 |
=================================================================
       1 ||    0 |    1 |   -- |   -- |   -- |   -- |   -- |   -- |
       2 ||   -- |    0 |   -- |   -- |   -- |   -- |   -- |   -- |
       3 ||   -- |   -- |    0 |   -- |   -- |   -- |   -- |   -- |
       4 ||    2 |    3 |   -- |    0 |   -- |   -- |   -- |    4 |
       5 ||   -- |   -- |   -- |   -- |    0 |   -- |   -- |   -- |
       6 ||   -- |   -- |   -- |   -- |   -- |    0 |   -- |   -- |
       7 ||   16 |   17 |   -- |   -- |   -- |   -- |    0 |    9 |
       8 ||    7 |    8 |   -- |   -- |   -- |   -- |   -- |    0 |
=================================================================
```

Оценка эффективности

При тестировании выполнялось 5 предварительных тестов, результаты которых не учитывались, после выполнялось 10 основных, по результатом которых составлялась таблицы.
В замер времени включался поиск пути наименьшей длины для всех пар вершин графа

Кол-во вершин	% от макс Кол-во ребер	Матричное представление		Списочное представлени		Сравнение памяти Матричное к Списочному	Сравнение времени Списочное к Матричному
		Память, байт	Время, сек	Память, байт	Время, сек		
50	10.0	10008	0.00037	4112	0.00034	58.91	7.55
50	20.0	10008	0.00049	7536	0.00049	24.70	0.14
50	27.5	10008	0.00057	9936	0.00052	0.72	8.70
50	30.0	10008	0.00055	10768	0.00056	-7.59	-0.85
50	50.0	10008	0.00055	16336	0.00054	-63.23	1.07
50	70.0	10008	0.00055	20480	0.00056	-104.64	-1.66
50	100.0	10008	0.00056	25328	0.00057	-153.08	-2.55
100	10.0	40008	0.00326	15904	0.00325	60.25	0.32
100	20.0	40008	0.00385	29504	0.00381	26.25	1.15
100	27.5	40008	0.00396	38944	0.00399	2.66	-0.82
100	30.0	40008	0.00400	41776	0.00399	-4.42	0.10
100	50.0	40008	0.00413	62624	0.00415	-56.53	-0.59
100	70.0	40008	0.00498	80160	0.00423	-100.36	15.09
100	100.0	40008	0.00417	100448	0.00425	-151.07	-1.74
300	10.0	360008	0.11931	138896	0.09955	61.42	16.56
300	20.0	360008	0.10216	262112	0.10226	27.19	-0.10
300	27.5	360008	0.10392	346912	0.11250	3.64	-8.26
300	30.0	360008	0.10633	374000	0.10651	-3.89	-0.17
300	50.0	360008	0.10494	566016	0.10615	-57.22	-1.16
300	70.0	360008	0.12450	724544	0.10689	-101.26	14.14
300	100.0	360008	0.10836	910272	0.10524	-152.85	2.88
500	10.0	1000008	0.46673	383824	0.46491	61.62	0.39
500	20.0	1000008	0.47347	726992	0.47388	27.30	-0.09
500	27.5	1000008	0.47245	961616	0.49094	3.84	-3.91
500	30.0	1000008	0.49435	1035984	0.47607	-3.60	3.70
500	50.0	1000008	0.47931	1570880	0.48083	-57.09	-0.32
500	70.0	1000008	0.48078	2010800	0.48584	-101.08	-1.05
500	100.0	1000008	0.48076	2523856	0.48185	-152.38	-0.23
700	10.0	1960008	1.29403	751168	1.28549	61.68	0.66
700	20.0	1960008	1.31410	1426816	1.29103	27.20	1.76
700	27.5	1960008	1.30619	1888112	1.31020	3.67	-0.31
700	30.0	1960008	1.32706	2036240	1.30671	-3.89	1.53
700	50.0	1960008	1.30680	3085504	1.30989	-57.42	-0.24
700	70.0	1960008	1.29428	3945744	1.30186	-101.31	-0.59
700	100.0	1960008	1.29820	4954144	1.30746	-152.76	-0.71
1000	10.0	4000008	3.75247	1530464	3.76208	61.74	-0.26
1000	20.0	4000008	3.75539	2906064	3.77798	27.35	-0.60
1000	27.5	4000008	3.80239	3852064	3.76917	3.70	0.87
1000	30.0	4000008	3.77805	4150624	3.75567	-3.77	0.59
1000	50.0	4000008	3.80429	6296208	3.79757	-57.40	0.18
1000	70.0	4000008	3.79044	8058096	3.82053	-101.45	-0.79
1000	100.0	4000008	3.83326	10107520	3.79665	-152.69	0.96

Память

Матричная реализация при количестве ребер меньше чем 27.5% от максимального количества требует больше затрат памяти. При этом при большей заполненности графа, списочная реализация требует больше затрат на память.
Максимально возможное количество ребер у графа из m вершин вычисляется по формуле $m(m-1)$

- При заполненности 10% матричная реализация требует на 62% больше памяти по сравнению с списочной, при заполненности 20% требует на 27% больше памяти
- При заполненности 27.5% затраты на память уравниваются
- При заполненности 50% матричная реализация требует на 57% меньше памяти по сравнению с списочной, при заполненности 70% требует на 101% меньше памяти памяти, при заполненности 100% требует на 152% меньше памяти

Время

Матричная и списочная реализации требуют практически одинаковые затраты на время выполнения, при количестве вершин больше 50, независимо от заполненности графа.

Практическое применение графов с алгоритмом Флойда-Уоршелла

Транспортная компания открывает свой первый филиал в новой области страны. Логистические сети ещё не настроены и компания теряет много денег на неоптимальные маршруты. Чтобы оптимизировать затраты компании, создается граф, в котором в качестве вершин выступают города, между которыми производится доставка, ребра это дороги между городами, вес ребер это затраты, необходимы для доставки товара из города А в город Б. С помощью алгоритма Флойда-Уоршелла ищутся кратчайшие пути для каждый пары городов. С помощью чего компания может оптимизировать доставку, сделав её максимально дешевой. Но так же к качестве весов ребер может выставить время доставки, чтобы максимально ускорить доставку, или же вычисляемый коэффициент показывающий соотношение времени доставки и её стоимости, чтобы получить усредненный результат по скорости и стоимости доставки.

Вывод

По результатам тестирования можно выделить следующее:

- Время поиска наименьшего пути для всех пар вершин графа затрачивает приблизительно одинаковое время для матричной и списочной реализаций
- Для разраженных графов (в нашем случае это означает, что количество ребер не более 27.5% от максимального количества) по использованию памяти эффективнее списочная релизация. При большей заполненности графа по памяти эффективнее матричная реализация
- Для решении задачи о поиске наименьшего пути между каждыми двумя вершинами удобнее использовать матричную реализацию, потому что алгоритм Флойда–Уоршелла в своей основе содержит динамическое программирование, которое построено на постепенном заполнении таблицы

Контрольные вопросы

1. Что такое граф?

Граф – конечное множество вершин и ребер, соединяющих вершины.

2. Как представляются графы в памяти?

(N – количество вершин в графе)

- **Матричное представление** – создается матрицы $dist_{N\text{x}N}$. Для пары индексов `i` и `j` : `dist[i][j]` равно либо стоимости перехода `i -> j` , либо признак отсутствия ребра (например очень большое число, 0, отрицательное число)
- **Списочное представление** – создается массив `adj_arr` размер N. Каждый `i` -й элемент представляет собой список из смежных вершины для вершины `i`

3. Какие операции возможны над графами?

- Добавление ребра между вершинами
- Удаление ребра между вершинами
- поиск кратчайшего пути от одной вершины к другой (если он есть);
- поиск кратчайшего пути от одной вершины ко всем другим;
- поиск кратчайших путей между всеми вершинами;
- поиск эйлерова пути (если он есть)
- поиск гамильтонова пути (если он есть).

4. Какие способы обхода графов существуют

- **DFS** – алгоритм начинает с выбранной вершины и идёт по одному направлению как можно глубже, пока не дойдёт до тупика, после чего возвращается назад и продолжает обход.

- BFS (поиск в ширину) – алгоритм начинает с определенной вершины и сначала выполняет обработку вершин смежных текущей, после чего переходит к одной из этих вершин

5. Где используются графовые структуры?

Графовые структуры применяются для описания объектов и связей между ними: в навигации и построении маршрутов, в компьютерных и социальных сетях, при анализе зависимостей в программах и планировании задач, а также в поисковых системах, ИИ, игры

6. Какие пути в графе Вы знаете?

- Путь – последовательность вершин, соединенных узлами
- Простой путь – путь, без повторения вершин и ребер
- Кратчайший путь – путь с минимальной суммой весов
- Цикл – замкнутый простой путь
- Эйлеров путь – путь, проходящий ровно один раз по каждому ребру
- Гамильтонов путь – путь, проходящий ровно один раз по каждой вершине

7. Что такое каркасы графа?

Каркас графа – подграф, который:

- содержит все вершины исходного графа
- является деревом